SGBDs, MySQL Workbench e SQL I

Fundamentos de Banco de Dados
Prof.ª Moniky Ribeiro
2026

O que vamos aprender

Reconhecer o papel de um SGBD e comparar sistemas populares.

Conectar o Workbench ao servidor e executar scripts.

Implementar tabelas e restrições.

01

Sistemas gerenciadores de bancos de dados

Conceitos, popularidade e escolha para o laboratório

Banco de dados, SGBD e cliente

Elemento	Função	Exemplo
Banco de dados	Coleção organizada de dados	empresa_aula
SGBD	Gerencia armazenamento e acesso	MySQL Server
Cliente	Envia comandos e mostra resultados	MySQL Workbench
Linguagem	Expressa definições e operações	SQL

O papel do SGBD

- Armazena os dados e permite consultá-los.
- Impõe restrições de integridade, como PK e FK.
- Controla transações e acesso concorrente.
- Gerencia usuários, permissões e recuperação.

SGBDs mais populares

Posição	SGBD	Modelo principal
1	Oracle	Relacional
2	MySQL	Relacional
3	Microsoft SQL Server	Relacional
4	PostgreSQL	Relacional
5	MongoDB	Documentos

DB-Engines, setembro de 2026: popularidade, não percentual de instalações.

Outros sistemas e critérios de escolha

Sistema	Característica útil para comparação
SQLite	Banco embutido, sem servidor separado
MariaDB	SGBD relacional com origem no MySQL
Redis	Armazenamento com modelo chave-valor

Fatores que definem a escolha do SGBD:
Modelo de dados, licença, suporte e ambiente

02

Primeiros passos no MySQL Workbench

Conexão, editor SQL e organização do trabalho

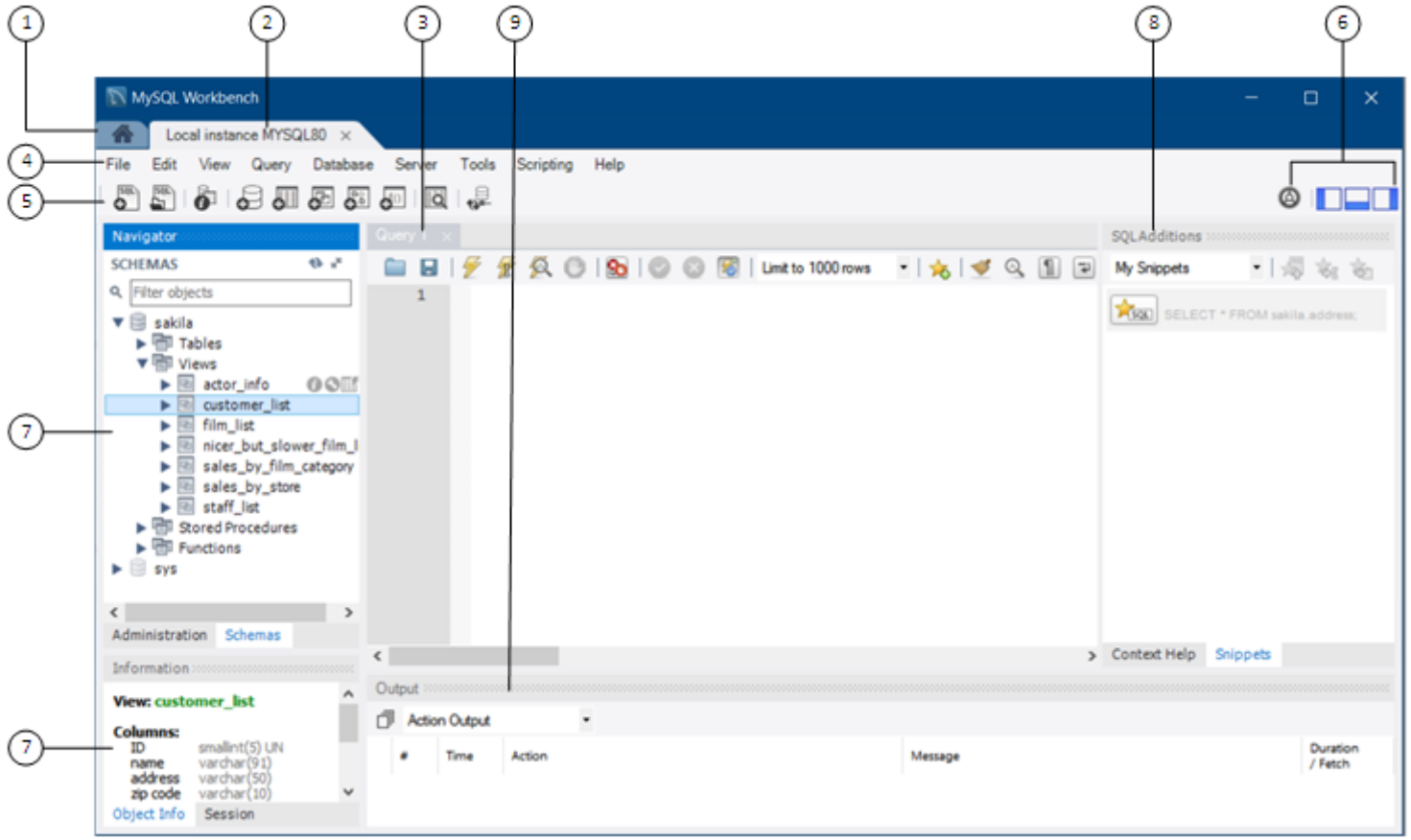
Como usar o workbench

- Instalar MySQL Community Server e MySQL Workbench. (Ou verificar se esta instalado).
- Configurar a senha do administrador durante a instalação.
- Confirmar que o serviço do servidor está iniciado.
- Usar uma conta de laboratório com as permissões necessárias.

Criando uma conexão

Campo ou ação	Configuração local de exemplo
MySQL Connections	Clicar no botão +
Connection Name	MySql_Connection
Connection Method	Standard (TCP/IP)
Hostname	127.0.0.1
Port	3306, se esta for a porta configurada
Username	Conta definida no laboratório
Test Connection	Informar a senha e testar (Senha já pode ter sido criada e você precisará resetar)

Áreas do Workbench



Interface ilustrativa da documentação oficial. A aparência depende da versão.

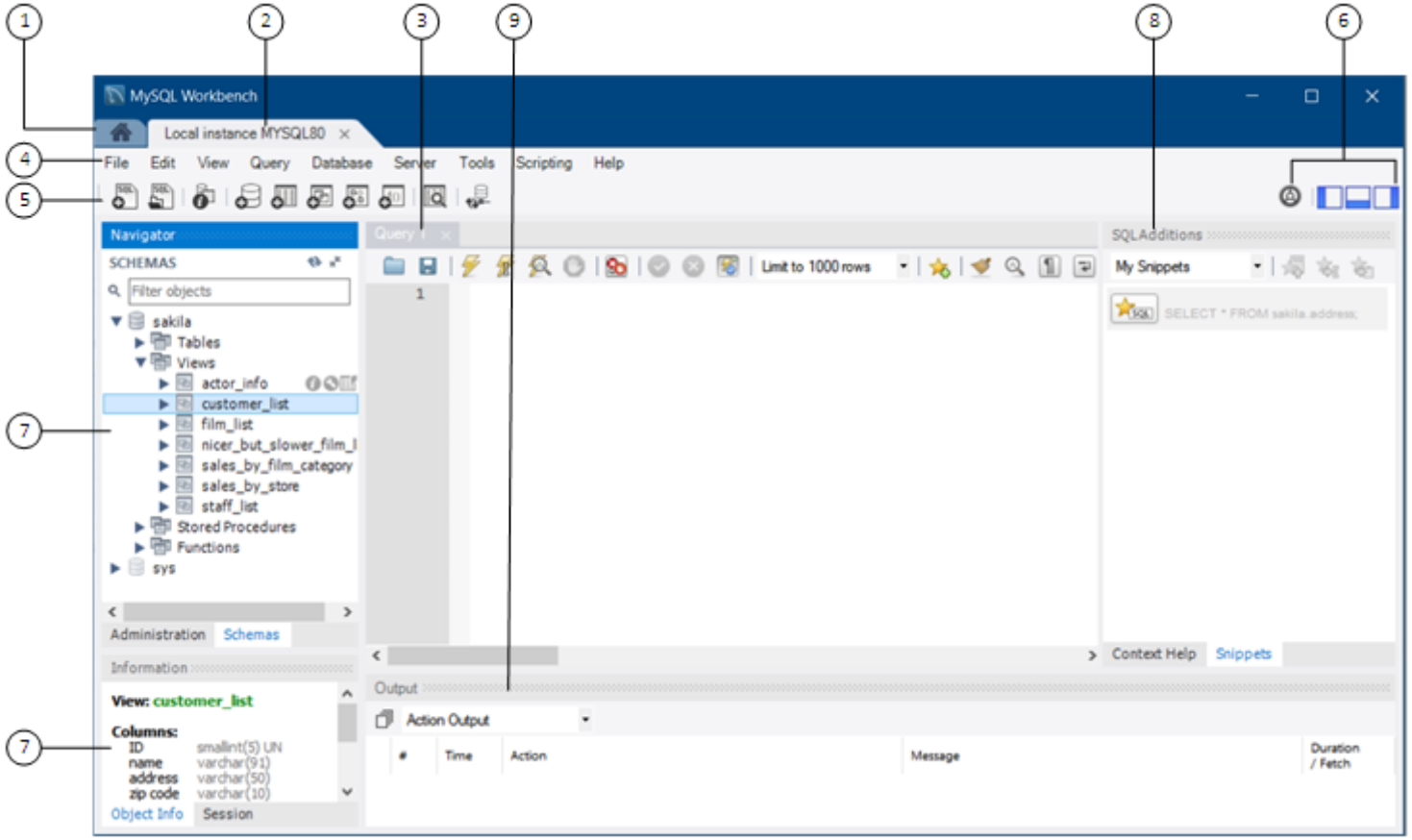
1 - Tela inicial

2 - Aba da conexão: Identifica o servidor conectado. Na imagem, a conexão se chama Local instance MySQL80.

3 - Aba de consulta: Contém o editor central, onde você escreve comandos SQL. Você pode abrir várias abas de consultas.

4 - Barra de menus: Reúne opções como abrir arquivos, editar, executar consultas e administrar o servidor.

Áreas do Workbench



Interface ilustrativa da documentação oficial. A aparência depende da versão.

5 - Barra de ferramentas principal: Oferece atalhos para criar ou abrir scripts e trabalhar com objetos do banco.

6 - Botões de exibir/ocultar painéis

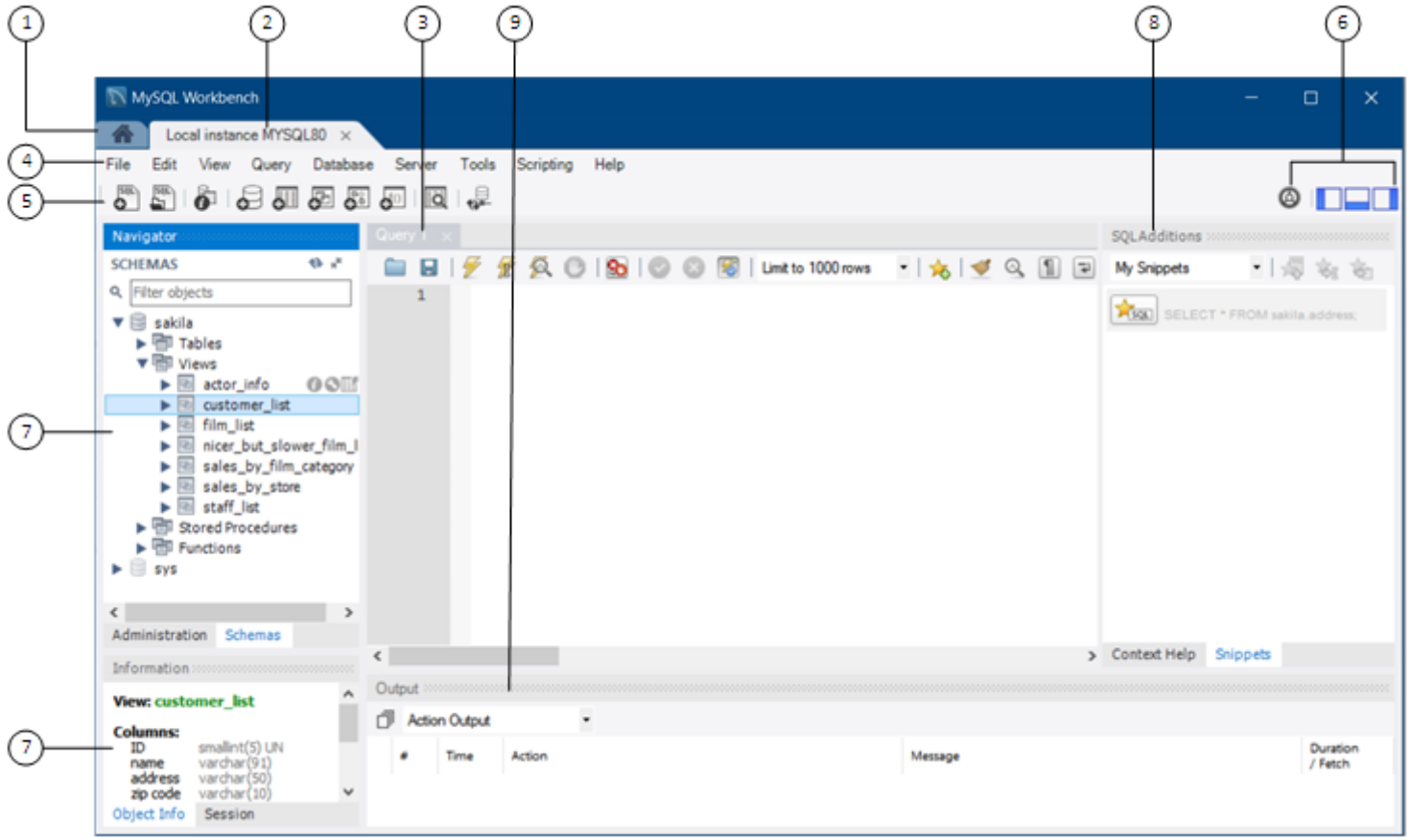
7 - Painel esquerdo — Navigator e Information : Em **Schemas**, mostra bancos, tabelas, views e outros objetos.

Na parte inferior, mostra detalhes do objeto selecionado, como suas colunas.

8 - Painel direito: SQL Additions

9 - Painel inferior — Output : Mostra o registro das execuções: comandos executados, mensagens de sucesso ou erro e duração.

Workbench



Interface ilustrativa da documentação oficial. A aparência depende da versão.

Schemas: bancos e objetos disponíveis.

Query Tab: comandos que vamos escrever.

Result Grid: linhas retornadas pela consulta.

Action Output: mensagens de execução e erros.

Execução e salvamento

Ação no Windows	Atalho ou procedimento
Executar comando sob o cursor	Ctrl + Enter
Executar seleção ou script	Ctrl + Shift + Enter
Abrir outra aba SQL	Ctrl + T
Salvar o script	Ctrl + S, extensão .sql
Atualizar Schemas	Botão de atualização do painel

Primeiro teste de conexão

- SELECT VERSION() AS versao_servidor;
- SELECT CURRENT_USER() AS conta;
- SELECT DATABASE() AS banco_atual;

O servidor devolve sua versão e a conta autenticada.
banco_atual pode ser NULL antes de selecionar um banco.

Criação e seleção do banco

```
CREATE DATABASE empresa_aula
CHARACTER SET utf8mb4;

USE empresa_aula;

SHOW TABLES;
```

CREATE DATABASE cria o banco.
USE seleciona o banco da sessão.
No MySQL, CREATE SCHEMA é sinônimo de CREATE DATABASE.

Erros comuns na primeira prática

Mensagem ou situação	Verificação
Cannot connect	Serviço, host e porta do servidor
Access denied	Conta, senha e permissões
No database selected	Executar USE empresa_aula
Table does not exist	Nome da tabela e banco selecionado
Unknown column	Nome da coluna e alias
Erro 1175: safe update	Rever filtro e uso da chave no WHERE

Script, modelo e backup

- Arquivo .sql: texto com os comandos escritos.
- Arquivo .mwb: modelo do Workbench
- Banco no servidor: tabelas, dados e outros objetos.
- Server > Data Export exporta estrutura e dados.

03

Definição dos dados em SQL

DDL e implementação do banco EMPRESA

Componentes da linguagem SQL

Família	Finalidade	Exemplos
DDL (Data Definition Language)	Definir objetos	CREATE, ALTER, DROP
DML (Data Manipulation Language)	Manipular e consultar dados	INSERT, UPDATE, DELETE, SELECT
DCL	Controlar privilégios	GRANT, REVOKE
TCL	Controlar transações	START TRANSACTION, COMMIT, ROLLBACK

Convenções que usaremos

Elemento	Formato no MySQL
Texto	Aspas simples retas: 'Engenharia'
Data	'1974-08-12'
Decimal	5558.00
Identificador	nome_da_coluna, sem aspas simples
Comentário	-- seguido de espaço, ou /* texto */
Fim de comando	Ponto e vírgula

Tipos de dados embutidos em SQL

Segundo C. J. Date: resumo de tipos do SQL padrão

Tipo	Uso	Exemplo
BOOLEAN	Valor lógico	ativo: TRUE ou FALSE
SMALLINT	Inteiro de menor faixa	dependentes: 3
INTEGER ou INT	Número inteiro	departamento: 10
BIGINT	Inteiro de maior faixa	eventos: 5000000000
NUMERIC(p,s)	Número decimal exato	salário: NUMERIC(10,2)
DECIMAL(p,s)	Número decimal exato	preço: DECIMAL(8,2)
FLOAT(p)	Número aproximado	medição científica

DECIMAL(10,2): 10 dígitos no total, dos quais 2 ficam após o ponto decimal.

Tipos de dados embutidos em SQL

Segundo C. J. Date: resumo de tipos do SQL padrão

Tipo	Uso	Exemplo
CHAR(n)	Texto de tamanho fixo	código: CHAR(4), '1163'
VARCHAR(n)	Texto de tamanho variável	nome: VARCHAR(60)
CLOB	Texto extenso	conteúdo de um relatório
BINARY(n)	Bytes de tamanho fixo	resumo criptográfico
VARBINARY(n)	Bytes de tamanho variável	conteúdo binário
BLOB	Conteúdo binário extenso	imagem ou arquivo

CLOB e BLOB representam sequências extensas de caracteres e bytes.

Tipos de dados embutidos em SQL

Segundo C. J. Date: resumo de tipos do SQL padrão

Tipo	Uso	Exemplo
DATE	Data	nascimento: '2008-05-25'
TIME	Horário	início da aula: '14:30:00'
TIMESTAMP	Data e horário	instante de um evento
INTERVAL	Intervalo temporal	duração de três dias

No MySQL: DATETIME também representa data e horário. TEXT substitui CLOB.
BOOLEAN é um alias de TINYINT(1). INTERVAL não é um tipo de coluna.

Tipos - Geral

Tipo	Uso	Exemplo
INT	Número inteiro	numero de departamento
CHAR(4)	Texto de tamanho fixo	identificador 1163
VARCHAR(60)	Texto de tamanho variável	nome
DECIMAL(10,2)	Número exato com duas casas	salario
DATE	Data	dtnasc
DATETIME	Data e horário	evento de auditoria

SQL: esquema relacional EMPRESA

FUNCIONARIO(ident, nome, sobrenome, endereco, dtnasc, salario, sexo, supident, dnumero)

DEPENDENTE(fident, nome, dt_nasc, sexo, relacionamento)

DEPARTAMENTO(numero, nome, gident, dtinicio)

LOCALIZACOES(dnumero, localizacao)

PROJETO(numero, nome, localizacao, dnumero)

TRABALHA_EM(pnumero, fident, horas)

A chave primária de cada relação está sublinhada.

CREATE TABLE: departamento

```
CREATE TABLE departamento (
  numero INT PRIMARY KEY,
  nome VARCHAR(60) NOT NULL,
  gident int,
  dtinicio DATE
);
```

OUTRA FORMA:

```
CREATE TABLE departamento (
  numero INT,
  nome VARCHAR(60) NOT NULL,
  gident int,
  dtinicio DATE,
CONSTRAINT pk_departamento PRIMARY KEY (numero)
);
```

CREATE TABLE: funcionario

Primeira etapa: definir os atributos e a chave primária.

```
CREATE TABLE funcionario (
  ident INT,
  nome VARCHAR(60) NOT NULL,
  sobrenome VARCHAR(60) NOT NULL,
  endereco VARCHAR(45),
  dtnasc DATE,
  salario DECIMAL(10,2),
  sexo CHAR(1),
  supident INT,
  dnumero INT NOT NULL,
  PRIMARY KEY (ident)
);
```

Tipos concretos

INT substitui TIPO_ID neste exemplo.

CHAR(1) permite códigos como F e M para sexo.

Sintaxe

NOT NULL vem antes da vírgula.

As chaves estrangeiras serão adicionadas depois.

Quais atributos são chaves estrangeiras?

```
supident INT,
dnumero INT NOT NULL

-- Referências desejadas:
-- supident: funcionario.ident
-- dnumero: departamento.numero
```

supident

Identifica o supervisor. Referencia outro funcionário da mesma tabela.

Pode ser NULL quando não há supervisor.

dnumero

Identifica o departamento do funcionário.

Neste modelo, todo funcionário deve informar um departamento.

Declarando as chaves estrangeiras

Adicione as restrições à tabela criada no slide 1, após criar DEPARTAMENTO.

```
ALTER TABLE funcionario
  ADD CONSTRAINT fk_func_supervisor
    FOREIGN KEY (supident)
    REFERENCES funcionario (ident),
  ADD CONSTRAINT fk_func_departamento
    FOREIGN KEY (dnumero)
    REFERENCES departamento (numero);
```

FOREIGN KEY

Indica a coluna que faz a referência.

REFERENCES indica a tabela e a coluna referenciadas.

Vírgulas e nomes

A vírgula separa as duas restrições.

CONSTRAINT permite nomear cada chave estrangeira.

NOT NULL e integridade referencial

As duas restrições verificam condições diferentes e podem atuar juntas.

dnumero INT NOT NULL

Exige que o funcionário informe um departamento.

FOREIGN KEY (dnumero) REFERENCES departamento (numero)

Exige que o departamento informado exista na tabela departamento.

Exemplo: o único departamento cadastrado tem numero = 4.

dnumero = 4: permitido
dnumero = 9: rejeitado pela chave estrangeira
dnumero = NULL: rejeitado pelo NOT NULL

Em que ordem criar as tabelas?

Crie as estruturas primeiro. Depois, adicione as referências que dependem delas.

```
-- 1. Crie departamento primeiro
CREATE TABLE departamento (
  numero INT,
  nome VARCHAR(60) NOT NULL,
  gident INT,
  dtinicio DATE,
  PRIMARY KEY (numero)
);

-- 4. Adicione a referência ao gerente
ALTER TABLE departamento
  ADD CONSTRAINT fk_dep_gerente
  FOREIGN KEY (gident)
  REFERENCES funcionario (ident);
```

Sequência completa

1. Crie departamento.
2. Crie funcionario (slide 1).
3. Adicione suas FKs (slide 4).
4. Adicione a FK do gerente.

Ao inserir dados

Departamento: gident = NULL.
Depois, insira o funcionário e atualize o gerente.

Algumas restrições para a tabela funcionário

```
  CONSTRAINT ck_salario CHECK (salario >= 0),
  CONSTRAINT fk_func_depto FOREIGN KEY (dnumero)
    REFERENCES departamento(numero),
  CONSTRAINT fk_func_super FOREIGN KEY (supident)
    REFERENCES funcionario(ident)
    ON DELETE SET NULL
);
```

FK impede referência a um registro inexistente.
CHECK impede salário negativo neste exemplo.

O significado das restrições

Restrição	Efeito
PRIMARY KEY	Identifica cada linha e impede duplicação e NULL
FOREIGN KEY	Mantém a referência entre tabelas
NOT NULL	Exige um valor na coluna
UNIQUE	Impede valores não nulos repetidos
DEFAULT	Fornece um valor quando a coluna é omitida
CHECK	Exige que a condição não seja falsa

Relação N:N: trabalha_em

```
CREATE TABLE trabalha_em (
  pnumero INT NOT NULL,
  fident CHAR(4) NOT NULL,
  horas DECIMAL(5,2) NOT NULL,
  PRIMARY KEY (pnumero, fident),
  FOREIGN KEY (pnumero) REFERENCES projeto(numero),
  FOREIGN KEY (fident) REFERENCES funcionario(ident),
  CHECK (horas >= 0)
);
```

A PK composta identifica a participação.
O mesmo par projeto–funcionário aparece uma única vez.

Criação em etapas

```
ALTER TABLE departamento
  ADD CONSTRAINT fk_depto_gerente
  FOREIGN KEY (gident) REFERENCES funcionario(ident);
```

1. Criar departamento e depois funcionario.
2. Acrescentar a FK do gerente.
3. Inserir departamentos com gident NULL.
4. Inserir funcionários e depois atribuir gerentes.

ALTER TABLE: alterações de estrutura

```
ALTER TABLE funcionario
  ADD COLUMN email VARCHAR(120);

ALTER TABLE funcionario
  MODIFY COLUMN nome VARCHAR(80) NOT NULL;

ALTER TABLE funcionario
  ALTER COLUMN sexo SET DEFAULT 'N';
```

ADD acrescenta uma coluna.
MODIFY redefine o tipo e as propriedades desejadas.
ALTER COLUMN pode mudar o valor padrão.

Renomear e remover elementos

```
ALTER TABLE projeto
  RENAME COLUMN localizacao TO cidade;
ALTER TABLE projeto
  RENAME COLUMN cidade TO localizacao;

ALTER TABLE funcionario DROP COLUMN email;
```

RENAME COLUMN está disponível no MySQL 8.0.
DROP COLUMN remove a coluna e seus dados.

Removendo uma tabela: DROP TABLE

O comando remove a estrutura da tabela e todos os registros que ela contém.

```
USE empresa;

DROP TABLE funcionario;

-- Confira as tabelas restantes
SHOW TABLES;
```

O que desaparece?

Colunas, registros, índices e restrições da tabela.

O banco empresa continua existindo.

Antes de executar

Confirme o banco e a tabela.

Para tabelas comuns, ROLLBACK não desfaz o DROP TABLE.

Usando IF EXISTS

Evite o erro de tabela inexistente ao executar novamente um script.

```
DROP TABLE IF EXISTS funcionario;

-- Exemplo com várias tabelas
DROP TABLE IF EXISTS
  dependente, trabalha_em;
```

Se a tabela existe

O MySQL remove a tabela e seus dados.

Se não existe

O comando não falha por esse motivo e gera uma nota.

IF EXISTS não evita erros de chaves estrangeiras.

Remoção e chaves estrangeiras

Remova primeiro as tabelas que dependem de outras.

```
-- funcionario referencia departamento
-- Sem outras tabelas dependentes:
DROP TABLE funcionario;
DROP TABLE departamento;
```

Por que essa ordem?

A referência parte de funcionario. Remover departamento primeiro pode gerar erro.

ON DELETE SET NULL

Essa regra atua na exclusão de registros com DELETE.

Ela não libera a remoção da tabela com DROP TABLE.

Quando há referências nos dois sentidos

Exemplo: funcionário pertence a departamento e departamento referencia seu gerente.

```
-- Consulte o nome real da restrição
SHOW CREATE TABLE departamento;

-- Se ela se chama fk_dep_gerente:
ALTER TABLE departamento
  DROP FOREIGN KEY fk_dep_gerente;

-- Sem outras tabelas dependentes:
DROP TABLE funcionario;
DROP TABLE departamento;
```

Quebre o ciclo

Remova a FK do gerente antes de remover as tabelas.

DROP FOREIGN KEY mantém a coluna gident e seus valores.

Outras dependências

DEPENDENTE e TRABALHA_EM devem ser removidas antes de FUNCIONARIO, se tiverem FKs para ela.

DROP TABLE ou DELETE?

Escolha o comando de acordo com o que deseja remover.

Remover um registro

DELETE FROM funcionario WHERE ident = 1;

Exclui o funcionário 1 e mantém a estrutura da tabela.

Remover a tabela inteira

DROP TABLE funcionario;

Exclui a estrutura e todos os registros. Para usá-la novamente, recrie a tabela.

DROP Data Base

Remover o esquema

DROP DATABASE empresa;

Remove o banco empresa, **todas as tabelas e todos os dados dele**.

Remover o esquema verificando existência

DROP DATABASE IF EXISTS empresa;

Confira o nome antes de executar.

Prática de DDL

Criar o banco empresa

Executar a criação das seis tabelas.

Conferir cada estrutura com DESCRIBE nome_da_tabela.

Verificar PKs e FKs com SHOW CREATE TABLE nome_da_tabela.

Comparar o resultado com o esquema relacional.

Atividade para aula da semana que vem

- Criação do banco de dados empresa - Individual
  - SCRIPT
- Criação do banco de dados do projeto individual - Individual
  - SCRIPT
- Trabalho de ocorrências aeronáuticas (PASSO 1 e 2) – Trio sorteado
  - MER adaptado
  - Modelo relacional
  - Relatório – Entendimento e explicação de possíveis adaptações

SUBA TUDO EM UM .ZIP para a atividade no SUAP.

Referências e arquivos de apoio

Elmasri e Navathe. Sistemas de Banco de Dados, 6ª edição.

Documentação oficial: dev.mysql.com/doc/refman/ e dev.mysql.com/doc/workbench/.

Popularidade: DB-Engines, recorte de setembro de 2026.